

Bias Audit of Pre-Trained Word Embeddings Using the Word Embedding Association Test (WEAT)

```
Copy of GenAI EX 2.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Connect

# Ex2: Bias Audit of Pre-Trained Word Embeddings Using WEAT
!pip install gensim
!pip install gensim

import gensim
import numpy as np
from numpy.linalg import norm

# -----
# Load Google News Word2Vec Model (2GB - takes time)
# -----
import gensim.downloader as api

print("loading pre-trained embedding...")
model = api.load('glove-gli-pigword-100')
print("Model loaded successfully")

# -----
# Define Target Word Sets
# -----
X = ["man", "male", "boy", "brother", "he", "him", "his"]
Y = ["woman", "female", "girl", "sister", "she", "her", "hers"]

# -----
# Define Attribute Word Sets
# -----
A = ["career", "business", "profession", "corporation", "salary", "office"]
B = ["family", "home", "children", "parents", "marriage", "house"]

# -----
# Helper Functions
# -----

def cosine_similarity(w1, w2):
    return np.dot(w1, w2) / (norm(w1) * norm(w2))

() Variables Terminal
```

```
Copy of GenAI EX 2.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Connect

def cosine_similarity(w1, w2):
    return np.dot(w1, w2) / (norm(w1) * norm(w2))

def association(w, A, B):
    return np.mean([cosine_similarity(model[w], model[a]) for a in A]) - \
        np.mean([cosine_similarity(model[w], model[b]) for b in B])

def weat_score(X, Y, A, B):
    return sum(association(x, A, B) for x in X) - \
        sum(association(y, A, B) for y in Y)

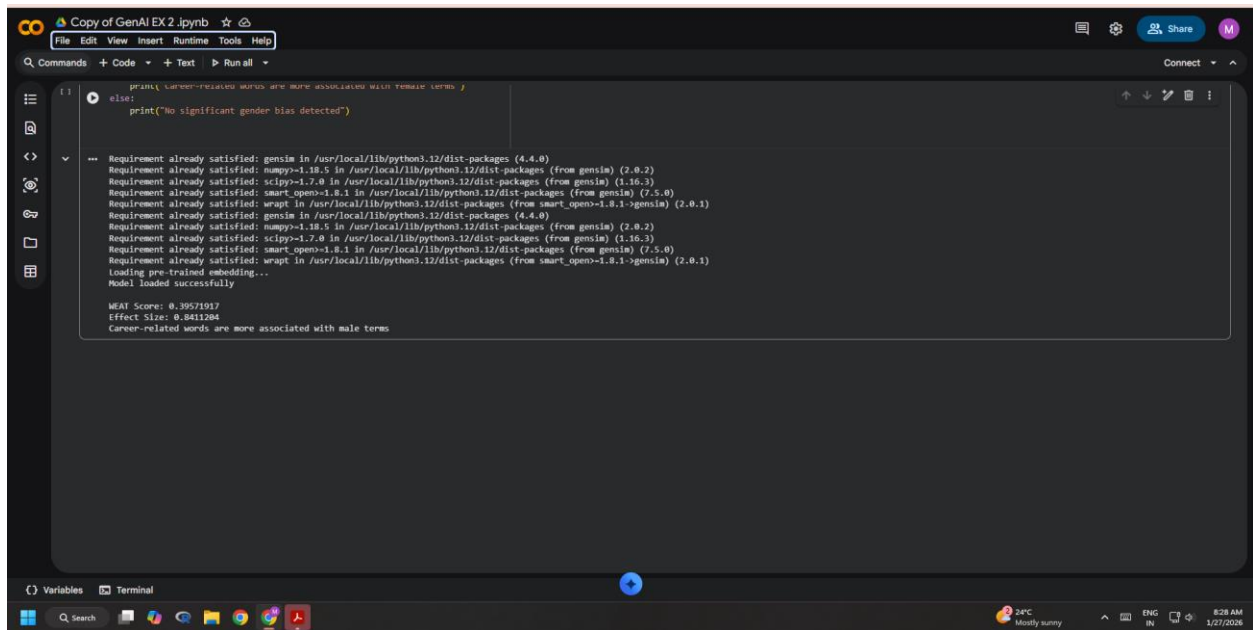
def effect_size(X, Y, A, B):
    assoc_X = [association(x, A, B) for x in X]
    assoc_Y = [association(y, A, B) for y in Y]
    mean_diff = np.mean(assoc_X) - np.mean(assoc_Y)
    std_dev = np.std(assoc_X + assoc_Y)
    return mean_diff / std_dev

# -----
# Compute WEAT Score and Effect Size
# -----
weat = weat_score(X, Y, A, B)
effect = effect_size(X, Y, A, B)

# -----
# Display Results
# -----
print("\nWEAT Score:", weat)
print("Effect Size:", effect)

if effect > 0:
    print("Career-related words are more associated with male terms")
elif effect < 0:
    print("Career-related words are more associated with female terms")
else:
    print("No significant gender bias detected")

() Variables Terminal
```



The screenshot shows a Jupyter Notebook titled "Copy of GenAI EX 2 .ipynb". The code in the cell is as follows:

```
print('Career-related words are more associated with female terms')  
else:  
    print("No significant gender bias detected")
```

The output of the code execution is:

```
--- Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)  
Requirement already satisfied: numpy--1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)  
Requirement already satisfied: scipy--1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)  
Requirement already satisfied: smart_open--1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)  
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open--1.8.1->gensim) (2.0.1)  
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)  
Requirement already satisfied: numpy--1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)  
Requirement already satisfied: scipy--1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)  
Requirement already satisfied: smart_open--1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)  
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open--1.8.1->gensim) (2.0.1)  
Loading pre-trained embedding...  
Model loaded successfully  
  
WEAT Score: 0.39571917  
Effect Size: 0.8411284  
Career-related words are more associated with male terms
```

The bottom of the interface shows a Windows taskbar with the date and time 8:28 AM 1/27/2026 and weather information: 24°C Mostly sunny.